

Onboarding Exercises: ROS 2 Learning Demos with PX4 and MAVROS 2

Objective

In this assignment, you will implement three learning demos using **ROS 2** and the **PX4 SITL** simulation environment connected via **MAVROS 2**. These exercises teach how to communicate with UAVs using ROS 2 topics and services, send velocity commands, and design control logic for autonomous motion and landing.

You will progressively develop:

1. A simple publisher–subscriber system (`talker--listener`);
2. Velocity control based on geometric paths;
3. A precision landing controller using feedback.

Environment Setup

This project uses a Dockerized PX4 environment with ROS 2 and MAVROS 2. Follow the steps carefully, noting which are done **outside** and which **inside** the container.

Outside Container (Host Ubuntu Linux)

Supported versions: Ubuntu 22.04 or 24.04

1. Install Docker

Install Docker via apt: <https://docs.docker.com/engine/install/ubuntu/>

Post-Install setup: <https://docs.docker.com/engine/install/linux-postinstall/>

2. Create Workspace Directory

```
mkdir -p /home/$(whoami)/Repos/src
```

Place your ROS 2 packages inside `~/Repos/src`, then clone the container image, and build it. It might take from 5-10 minutes.

```
cd /home/$(whoami)/Repos
git clone git@github.com:OpenHORYZN/base_image.git
cd base_image
./build_container.sh
```

Then start your container in the background. (You need to restart this after any reboot)

```
cd /home/$(whoami)/Repos/base_image
docker compose up -d
```

3. Install GStreamer and Dependencies

```
sudo apt install gstreamer1.0-plugins-bad gstreamer1.0-libav
gstreamer1.0-gl -y &&
sudo apt install libfuse2 libxcb-xinerama0 libxkbcommon-x11-0
libxcb-cursor-dev -y
```

4. Install QGroundControl

```
wget https://d176tv9ibo4jno.cloudfront.net/latest/QGroundControl.AppImage &&
chmod +x QGroundControl.AppImage &&
./QGroundControl.AppImage
```

Inside Container (PX4 + ROS 2 Environment)

The Docker container `px4-environment` includes ROS 2, MAVROS 2, PX4 SITL, and Gazebo. All ROS 2 commands must be run **inside** this container.

Step 1: Enter the Container

```
docker exec -it px4-environment bash
```

Expected prompt:

```
root@px4-container:/#
```

Step 2: Build the ROS 2 Workspace

```
cd /ros2_ws
colcon build
source install/setup.bash
```

Tip: Always re-source after every build.

Step 3: Launch PX4 SITL with Gazebo

```
cd /PX4-Autopilot
make px4_sitl gz_x500_lawn
```

You should see:

```
INFO [simulator] Simulator connected on TCP port 4560.
INFO [commander] Takeoff detected
```

Step 4: Open a Second Terminal for MAVROS 2

From the host:

```
docker exec -it px4-environment bash
```

Inside the container:

```
source /opt/ros/$ROS_DISTRO/setup.bash
ros2 launch mavros px4.launch fcU_url:=udp://:14540@
```

Check connectivity (on a separate terminal inside the container):

```
ros2 topic list
```

Expected topics:

```
...
/mavros/local_position/pose
/mavros/setpoint_velocity/cmd_vel_unstamped
/mavros/state
```

Step 5: Open a Third Terminal for Your Nodes

```
docker exec -it px4-environment bash
cd /ros2_ws
source install/setup.bash
ros2 run <Your-Package> <node_name>
```

Notes:

- All ROS 2 operations must be done inside the container.
- Keep PX4 SITL, MAVROS 2, and your nodes in separate terminals.

Install these ROS 2 Packages

```
sudo apt install
  ros-${ROS_DISTRO}-mavros
  ros-${ROS_DISTRO}-mavros-extras
  ros-${ROS_DISTRO}-geometry-msgs
  ros-${ROS_DISTRO}-tf2-ros
  ros-${ROS_DISTRO}-rqt-graph
  ros-${ROS_DISTRO}-rqt-topic -y
```

Task 1: Talker–Listener (Beginner)

Goal: Implement two ROS 2 nodes that exchange messages via a topic. This introduces the ROS 2 publish–subscribe model and teaches how nodes communicate asynchronously.

Theory: In ROS 2, data exchange between nodes happens through *topics*. A Publisher sends messages of a given type to a topic, while a Subscriber listens to that same topic and receives messages.

Step 1: Create the Package

From inside your ROS 2 workspace (container) (/ros2_ws/src), create a Python package:

```
cd /ros2_ws/src
ros2 pkg create --build-type ament_python talker_listener_demo
```

This creates:

```
/ros2_ws/src/talker_listener_demo/
package.xml
setup.py
talker_listener_demo/
  __init__.py
```

Step 2: Add the Node Scripts

Go into the source folder:

```
cd /ros2_ws/src/talker_listener_demo/talker_listener_demo
```

Create two Python files:

```
touch talker.py listener.py
```

talker.py:

```

import rclpy
from rclpy.node import Node
from std_msgs.msg import String

class Talker(Node):
    def __init__(self):
        super().__init__('talker')
        self.pub = self.create_publisher(String, 'chatter', 10)
        self.timer = self.create_timer(1.0, self.timer_callback)
        self.count = 0

    def timer_callback(self):
        msg = String()
        msg.data = f'Hello PX4 #{self.count}'
        self.pub.publish(msg)
        self.get_logger().info(f'Publishing: "{msg.data}"')
        self.count += 1

def main(args=None):
    rclpy.init(args=args)
    node = Talker()
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()

```

listener.py:

```

import rclpy
from rclpy.node import Node
from std_msgs.msg import String

class Listener(Node):
    def __init__(self):
        super().__init__('listener')
        self.create_subscription(String, 'chatter', self.callback, 10)

    def callback(self, msg):
        self.get_logger().info(f'I heard: "{msg.data}"')

def main(args=None):
    rclpy.init(args=args)
    node = Listener()
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()

```

Step 3: Add Entry Points

Edit `setup.py` and add this inside the `entry_points` block:

```
entry_points={
    'console_scripts': [
        'talker = talker_listener_demo.talker:main',
        'listener = talker_listener_demo.listener:main',
    ],
},
```

Step 4: Build and Source the Workspace

Go back to the workspace root and build:

```
cd /ros2_ws
colcon build
```

After every successful build, source the workspace:

```
source install/setup.bash
```

Tip:

- Build after adding or modifying Python/C++ files or `setup.py`.
- Source after every new build or when opening a new terminal.

Step 5: Run and Test

Open two terminals (both must source the workspace before running):

Terminal 1 – Talker:

```
cd /ros2_ws
source install/setup.bash
ros2 run talker_listener_demo talker
```

Terminal 2 – Listener:

```
cd /ros2_ws
source install/setup.bash
ros2 run talker_listener_demo listener
```

Expected output:

```
[INFO] [talker]: Publishing: "Hello PX4 #0"
[INFO] [listener]: I heard: "Hello PX4 #0"
[INFO] [talker]: Publishing: "Hello PX4 #1"
[INFO] [listener]: I heard: "Hello PX4 #1"
```

Tips:

- You can inspect active topics using: `ros2 topic list`

- To check message flow: `ros2 topic echo /chatter`
- Use `Ctrl + C` to stop nodes cleanly.
- Always work inside the Docker container

Task 2: Shape-Based Velocity Control (Intermediate)

Goal: Command the UAV to fly continuous geometric trajectories — specifically a **circle**, an **ellipse**, and a **figure-eight** (∞) — using velocity commands published to `/mavros/setpoint_velocity/cmd_vel`. This exercise demonstrates how position trajectories can be differentiated to generate smooth velocity profiles for OFFBOARD control.

Theory

In OFFBOARD mode, PX4 requires continuous velocity updates (at least 2 Hz) through MAVROS. The drone's position evolves as:

$$\dot{x} = v_x, \quad \dot{y} = v_y$$

If you define a desired path $(x(t), y(t))$, you can obtain the corresponding velocity commands by differentiating with respect to time:

$$v_x = \frac{dx}{dt}, \quad v_y = \frac{dy}{dt}$$

This ensures the UAV follows the shape smoothly and continuously.

Step 1: Define the Trajectories

Let R be the radius or scale, and ω the angular frequency (rad/s). Keep altitude constant (e.g., $z = 2.0$ m).

1. Circle

$$\begin{aligned} x(t) &= R \cos(\omega t), & y(t) &= R \sin(\omega t) \\ v_x &= -R\omega \sin(\omega t), & v_y &= R\omega \cos(\omega t) \end{aligned}$$

2. Ellipse

$$\begin{aligned} x(t) &= a \cos(\omega t), & y(t) &= b \sin(\omega t) \\ v_x &= -a\omega \sin(\omega t), & v_y &= b\omega \cos(\omega t) \end{aligned}$$

3. Figure-Eight (Infinity)

$$\begin{aligned} x(t) &= R \sin(\omega t), & y(t) &= R \sin(\omega t) \cos(\omega t) \\ v_x &= R\omega \cos(\omega t), & v_y &= R\omega [\cos^2(\omega t) - \sin^2(\omega t)] \end{aligned}$$

These velocity components are then sent directly as commands to PX4.

Step 2: Node Structure (Python Skeleton)

```
import rclpy
from rclpy.node import Node
from geometry_msgs.msg import TwistStamped
import math, time

class ShapeController(Node):
    def __init__(self):
        super().__init__('shape_controller')
        # Publisher for velocity commands
        self.pub = self.create_publisher(
            TwistStamped,
            '/mavros/setpoint_velocity/cmd_vel',
            10
        )
        # Timer for periodic publishing
        self.timer = self.create_timer(0.05, self.timer_callback)
        self.start_time = time.time()

    def timer_callback(self):
        t = time.time() - self.start_time

        # Compute v_x, v_y according to desired shape equations
        vx = ...
        vy = ...
        vz = 0.0

        # Fill and publish message
        msg = TwistStamped()
        msg.twist.linear.x = vx
        msg.twist.linear.y = vy
        msg.twist.linear.z = vz
        self.pub.publish(msg)

def main(args=None):
    rclpy.init(args=args)
    node = ShapeController()
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()
```

Step 3: Control Setup

Switch PX4 into OFFBOARD mode and arm before running:

```
ros2 service call /mavros/set_mode mavros_msgs/srv/SetMode
"{custom_mode: 'OFFBOARD'}"
ros2 service call /mavros/cmd/arming mavros_msgs/srv/CommandBool
"{value: true}"
```

Tips:

- Always publish at a rate above 2 Hz, otherwise PX4 will exit OFFBOARD mode.
- Keep altitude constant by holding $v_z = 0.0$.
- Use `rqt_plot` to visualize $x(t)$, $y(t)$ and verify shape patterns.

- The derivatives correspond to instantaneous velocity directions — physically aligning with the tangent of the path.

Expected Outcome: The UAV should fly smooth continuous patterns (circle, ellipse, or infinity) in Gazebo, maintaining a fixed altitude and steady speed.

Task 3: Precision Landing (Advanced)

Goal: Design a feedback controller that autonomously aligns the UAV with a landing target and performs a smooth descent using visual feedback from an **ArUco marker**.

This exercise combines visual perception (OpenCV) with PX4 control via MAVROS 2, teaching how to integrate sensor data into a control loop.

Important Remark

When working with PX4 SITL simulation, **QGroundControl (QGC) must be running at all times**. QGC ensures that PX4 receives heartbeat messages and maintains proper communication during OFFBOARD operations. Launch QGC on your host before starting any of the simulation or ROS 2 nodes.

Step 1: Launch Simulation with ArUco World

Inside the Docker container, run the PX4 SITL simulation with a world that includes a visible ArUco marker on the ground:

```
cd /PX4-Autopilot
make px4_sitl gz_x500_aruco
```

This launches Gazebo (gz-sim) with an ArUco tag placed on the landing pad. Keep this process running in a dedicated terminal.

Step 2: ArUco Tag Detection with OpenCV

Install OpenCV inside the container (if not already installed):

```
apt update && apt install python3-opencv -y
```

OpenCV provides built-in ArUco detection utilities. The detection process estimates the 2D pixel coordinates or full 3D pose of the marker relative to the camera frame. The 3D pose can then be converted into local (x_t, y_t, z_t) target coordinates for precision landing.

Example Detection Snippet:

```
import cv2
import cv2.aruco as aruco

# Load dictionary and parameters
aruco_dict = aruco.getPredefinedDictionary(aruco.DICT_4X4_50)
parameters = aruco.DetectorParameters()

cap = cv2.VideoCapture(0) # or simulated camera topic
```

```

while True:
    ret, frame = cap.read()
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    corners, ids, rejected = aruco.detectMarkers(gray, aruco_dict,
                                                parameters=parameters)

    if ids is not None:
        frame = aruco.drawDetectedMarkers(frame, corners, ids)
        print("Detected marker IDs:", ids)

    cv2.imshow('Aruco Detection', frame)
    if cv2.waitKey(1) == 27: # ESC to exit
        break

```

To estimate the tag's position in the camera frame (for feedback control):

```

rvecs, tvecs, _ = aruco.estimatePoseSingleMarkers(corners,
                                                markerLength=0.15,
                                                cameraMatrix=K,
                                                distCoeffs=D)

```

Where:

- `markerLength` — actual side length of the tag (in meters)
- `K` — camera intrinsic matrix
- `D` — distortion coefficients

Publish the estimated (x_t, y_t, z_t) as a ROS 2 topic, e.g.:

```

/target_position geometry_msgs/msg/Point

```

Step 3: Precision Landing Control Logic

Your control node should subscribe to:

```

/mavros/local_position/pose
/target_position

```

Compute positional errors:

$$\Delta x = x_t - x_c, \quad \Delta y = y_t - y_c, \quad \Delta z = z_c - z_t$$

Then generate velocity commands proportional to these errors:

$$v_x = k_p \Delta x, \quad v_y = k_p \Delta y, \quad v_z = -k_p \Delta z$$

and publish them to:

```

/mavros/setpoint_velocity/cmd_vel

```

Step 4: Controller Skeleton (Python)

```
import rclpy
from rclpy.node import Node
from geometry_msgs.msg import TwistStamped, Point

class PrecisionLanding(Node):
    def __init__(self):
        super().__init__('precision_landing')
        self.vel_pub = self.create_publisher(TwistStamped,
                                              '/mavros/setpoint_velocity/cmd_vel',
                                              10)

        self.create_subscription(Point, '/target_position',
                                  self.target_cb, 10)
        self.create_subscription(Point, '/mavros/local_position/pose',
                                  self.pose_cb, 10)

        self.target = Point()
        self.pose = Point()
        self.kp = 0.3

    def target_cb(self, msg):
        self.target = msg

    def pose_cb(self, msg):
        self.pose = msg.position

    def timer_callback(self):
        err_x = self.target.x - self.pose.x
        err_y = self.target.y - self.pose.y
        err_z = self.pose.z - self.target.z
        msg = TwistStamped()
        msg.twist.linear.x = self.kp * err_x
        msg.twist.linear.y = self.kp * err_y
        msg.twist.linear.z = -self.kp * err_z
        self.vel_pub.publish(msg)
```

Step 5: Tips and Expected Behavior

- The UAV should hover above the marker, center itself based on visual feedback, and then descend gradually.
- Limit descent rate (e.g., $|v_z| < 0.5$ m/s) to prevent oscillations.
- Use `rqt_plot` to visualize Δx , Δy , and Δz convergence.
- When $z < 0.1$ m, call:

```
ros2 service call /mavros/cmd/arming mavros_msgs/srv/CommandBool
"{value: false}"
```

to disarm safely after landing.

Expected Outcome: The UAV autonomously detects the ArUco marker, aligns itself over the center, and performs a smooth precision landing on the marked pad while QGroundControl remains connected in the background.